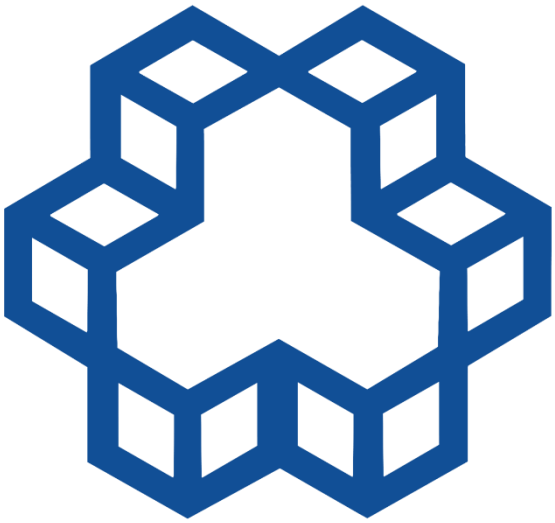
K. N. Toosi University of Technology

Faculty of Computer Engineering

Spring Semester
1404–1405



Course Project Report:
Compiler Design

Project Title:
CC-IDE Code-Aware Front-End & Middle-End Program Analyzer

Instructor:
Dr. Alaeiyan

Repository:
GitHub Repository

Project Developer & Responsibilities:

Name	Student ID
Ali Kashi Pazha	40224641

The development of the CC-IDE Analyzer was independently executed by Ali Kashi Pazha, who was solely responsible for the complete implementation of both the frontend and middle-end compiler passes. The areas of ownership and developed features include:

- **Ali Kashi Pazha**'s *Area of Ownership:*
    - Lexical Analyzer (DFA Lexer)
    - LL(k) Recursive-Descent Parser
    - AST-guided Syntax Highlighter
    - AST Nodes hierarchy
    - Hierarchical Symbol Table & Semantic Type Checker
    - Control Flow Graph (CFG) & Call Graph Builders
    - Dominator Tree (Lengauer-Tarjan)
    - Static Single Assignment (SSA) Form
    - DevOps & Infrastructure (Docker Containerization, GitHub Actions CI/CD Pipeline)
    - Automated Testing Infrastructure (Coverage configuration)
    - Automatic Language Detection.
    - Interactive CLI REPL.

Table of Contents:

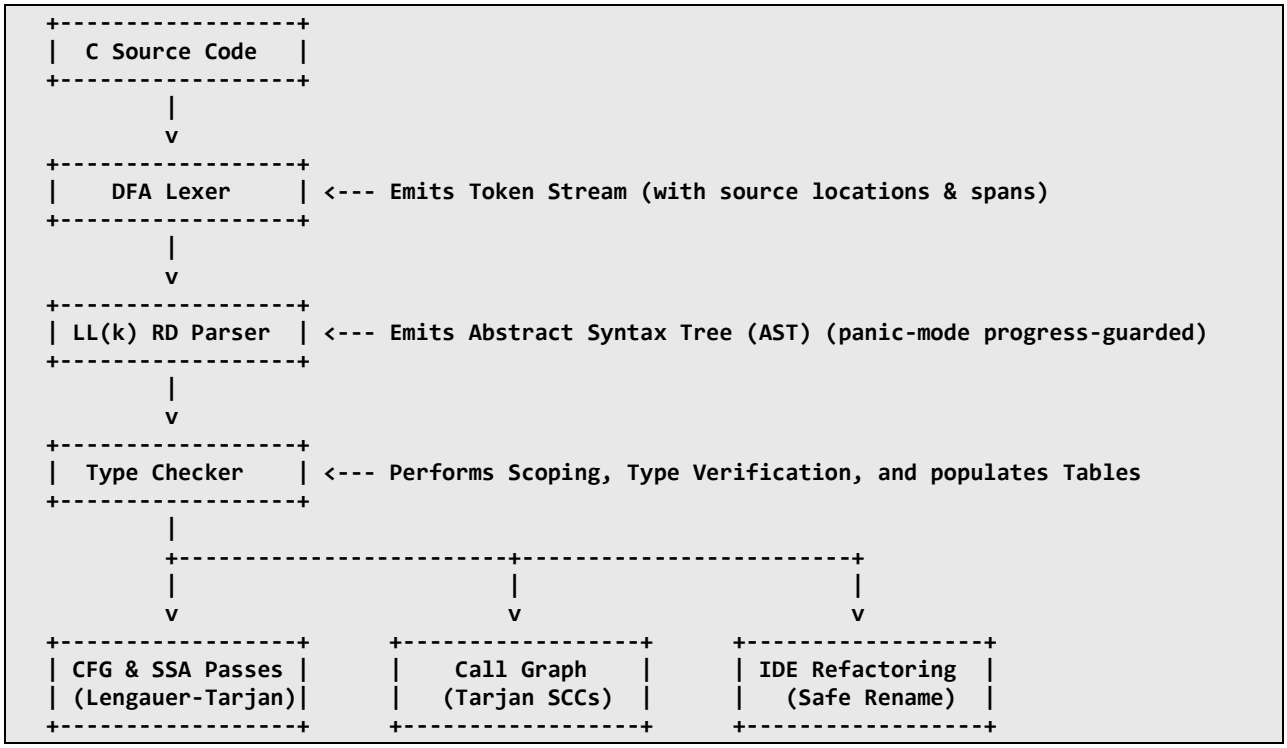
1. Executive Summary & Architecture Overview

This report documents the design and native Python implementation of a code-aware compiler front-end and middle-end static analysis suite optimized for a statically-typed subset of the C programming language. Developed entirely from first principles without relying on third-party parser generators, the system provides a robust pipeline of analyses that begins with raw source text and produces structured, context-sensitive program representations.

The primary architectural components include a hand-written DFA Lexer, an LL(k) Recursive-Descent Parser with panic-mode progress-tracking recovery, a hierarchical lexical Scope Resolver, a C-style Type Checker, a Control Flow Graph (CFG) generator, a program-wide Call Graph, and a safe Rename Refactoring engine.

1.1. System Pipeline and Data Flow

The structural data flow is designed with a decoupling of stages (separation of concerns), enabling stable and independent development of each analysis pass:



---

2. Formal Grammar Specification (EBNF)

To enable top-down, non-backtracking syntactic parsing, we designed and refined an unambiguous Extended Backus-Naur Form (EBNF) grammar. It accounts for function declarations, pointer notations, structure definitions, local scopes, standard branching, and expression priorities:

```
program        ::= declaration* EOF
declaration    ::= function_decl | var_decl | struct_decl
function_decl  ::= type_spec IDENT '(' param_list? ')' block
param_list     ::= param (',' param)*
param          ::= type_spec IDENT
type_spec      ::= ('struct' IDENT | 'int' | 'float' | 'char' | 'void' | 'double') '*'*
block          ::= '{' statement* '}'
statement      ::= if_stmt | while_stmt | for_stmt | return_stmt | expr_stmt | block |var_decl
if_stmt        ::= 'if' '(' expr ')' statement ('else' statement)?
while_stmt     ::= 'while' '(' expr ')' statement
for_stmt       ::= 'for' '(' expr_stmt expr_stmt expr? ')' statement
return_stmt    ::= 'return' expr? ';'
expr_stmt      ::= expr? ';'
var_decl       ::= type_spec IDENT ('=' expr)? ';'
struct_decl    ::= 'struct' IDENT '{' var_decl* '}' ';'
expr           ::= assignment
assignment     ::= IDENT ('='|'+='|'-='|'*=') assignment | logical_or
logical_or     ::= logical_and ('||' logical_and)*
logical_and    ::= equality ('&&' equality)*
equality       ::= relational (('=='|'!=') relational)*
relational     ::= additive (('<'|'>'|'<='|'>=') additive)*
additive       ::= multiplicative (('+'|'-') multiplicative)*
multiplicative ::= unary (('*'|'/'|'%') unary)*
unary          ::= ('-'|'!'|'&'|'*') unary | postfix
postfix        ::= primary ('[' expr ']' | '(' arg_list? ')' | '.' IDENT | '->' IDENT)*
primary        ::= INT | FLOAT | STRING | CHAR | IDENT | '(' expr ')'
arg_list       ::= expr (',' expr)*
```

---

3. Lexical Analysis (Phase 1)

3.1. Hand-Written DFA Design

The Lexical Analyzer (**lexer.py**) was designed and simulated as a hand-crafted Deterministic Finite Automaton (DFA) that translates raw character streams into typed Token objects. It is completely independent of external regex-splitting libraries for token boundary resolution.

Key tokenization rules include:

- **Longest-Match (Maximal Munch) Rule:** The scanner evaluates candidate lexemes iteratively. The lexer consumes characters as long as they form a valid token (e.g., matching **->** as a single **OP_ARROW** operator instead of **-** followed by **>**).
- **Priority Rule:** Fixed keywords take priority over identifiers. An identifier buffer matching "while" is returned as **KW_WHILE**, not **IDENT**.
- **Position Tracking:** Each token strictly records its starting line, starting column, absolute start_pos, and end_pos. This forms the foundation for context-aware syntax highlighting, downstream error reporting, and IDE refactoring.
- **Literal Scanning:** Handles decimal integers (42), hexadecimal constants (0xFF), binary integers (0b1010), and floating-point constants including scientific notation (1.0e-5) and dot-leading formats (.5).

3.2. Lexical Error Recovery

The Lexer implements a non-crashing recovery model and is designed to never crash. Unrecognized characters (e.g. @ or $) generate an INVALID token, the exact position is logged, and scanning immediately resumes on the next character. Additionally, the lexer detects and reports **unterminated string literals** (terminated by newline before a closing quote) and **unterminated block comments** (reaching EOF without */), capturing them as structured INVALID tokens to let the diagnostic system report accurate issues.

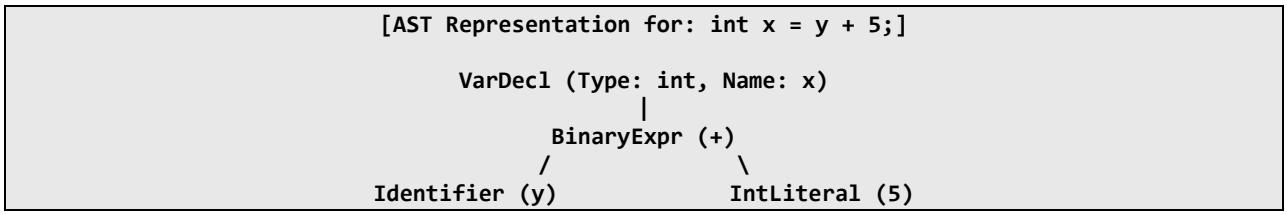
---

4. Syntactic Analysis & AST Construction (Phase 1)

4.1. LL(k) Parser & AST Implementation

The Parser (**parser.py**) consumes the token stream generated by the Lexer. It is implemented as a hand-written **LL(k) Recursive-Descent Parser** where each non-terminal in the EBNF grammar maps to a dedicated parsing method. Since the EBNF grammar is left-recursion-free and uses iteration (*) for left-associative operators, it fits the top-down parsing model perfectly.

The parser produces a nested **Abstract Syntax Tree (AST)** composed of strongly-typed node classes (**ast_nodes.py**). The AST is designed with accept hooks to support the **Visitor Pattern**, facilitating clean traversals for type checking, formatting, and tree-based code highlighted queries. Every node retains its starting SourceLocation for compile-time and run-time error tracing.



4.2. Panic-Mode Recovery with Progress-Tracking

To survive syntax errors and prevent immediate compilation failures, the parser implements **panic-mode recovery** (_synchronize). When a syntax mismatch occurs (e.g., a missing semicolon or parenthetical), a ParseError is thrown, unwinding the call stack to a safe synchronization boundary (like statement blocks, semicolons, or global declarations).

- **Progress-Tracking Infinite Loop Guard:** Standard panic-mode synchronizers can fall into infinite loops if they encounter a token that matches a synchronization starter keyword (e.g., for at the global level, which is a statement and not a valid declaration). To prevent this, the parser tracks last_error_index. If an error occurs at the same token index consecutively, the parser **forces an advance of at least one token** to guarantee forward progress toward EOF. This solves the infinite loop while maintaining accurate semicolon and brace

synchronization, ensuring the compiler continues processing the rest of the file.

---

5. Semantic Analysis & Type System (Phase 2)

5.1. Hierarchical Symbol Table & Lexical Scoping

The Symbol Table (**symbol_table.py**) manages name binding across nested scopes in a hierarchical tree. This mirrors the block structures of the program, consisting of a global scope, function-body scopes, local block scopes (within curly braces), and dedicated struct member scopes:

```
Global Scope
  ├── Struct: Point -> Member Scope: { x: int, y: int }
  ├── Function: calculate -> Signature: (int) -> int
  └── Block Scope (inside calculate)
        ├── Parameter: factor -> int
        └── Variable: p -> struct Point
```

Each Symbol records its name, kind (variable, function, parameter, type, struct_member), type expression string, definition location, and usage references. Lexical name resolution walks the scope chain upwards from the local scope to the global scope. If an identifier is referenced but not found, an "Undefined symbol" error is reported. If an inner variable hides a variable defined in an outer scope, a "**variable shadows outer declaration**" warning is emitted.

5.2. Static Type Checking & Safety Verification

The TypeChecker (**type_checker.py**) traverses the AST using the Visitor pattern in two passes:

- **Pass 1 (Declaration Scan):** Scans all global structs and function headers to register signatures. This allows **forward references** (e.g., recursive calls or using a struct pointer declared later) to resolve natively without ordering constraints.
- **Pass 2 (Resolution and Type Enforcement):** Traverses blocks and expressions. It enforces static C typing rules:
    - **Implicit Widening/Precision Loss Warning:** Emitted when implicitly assigning a float/double expression to an integer variable (e.g. int x = 3.14;).
    - **Pointer Compatibility:** Ensures a pointer type (T*) can only accept compatible pointer types or explicit type coercions, reporting an error on direct integer assignments (e.g. char* s = 42;) or mismatching pointer types.
    - **Struct Subscripting Accesses:** Verifies member accesses. For dot (.), the LHS must be a struct. For arrow (->), the LHS must be a

struct pointer. The field is searched inside the struct's registered member scope; missing fields trigger an "**Undefined member**" error.

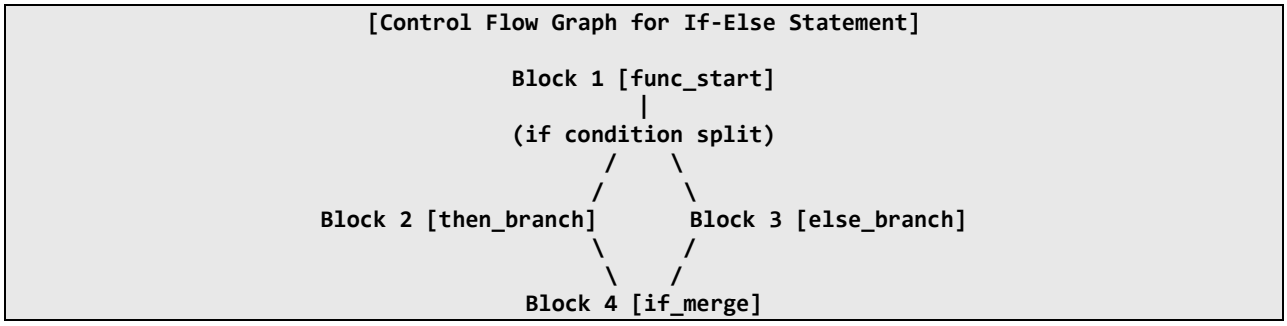
- **Uninitialized Read Tracking:** Performs linear initialization tracking. If a variable is read in an expression before being assigned to (via an initializer or assignment statement), a "**may be used uninitialized**" warning is reported.
- **Unused Variable Collection:** At the end of block scopes, any variable with **is_used == False** triggers an "**unused variable**" information warning.

---

6. Middle-End Program Analysis (Phase 3)

6.1. Control Flow Graph (CFG) Construction

The CFGBuilder (**cfg.py**) constructs a Control Flow Graph for every function body. It breaks statements into sequential, non-branching **Basic Blocks** (BasicBlock).



The CFG represents branches and loops explicitly as directed edges:

- An if-else statement splits the current basic block into two successors (**then_branch** and **else_branch**), which then merge back into an **if_merge** block.
- A while loop connects the preceding block to a loop condition block (**while_cond**), which branches to the loop body (**while_body**) and the loop merge block (**while_merge**). The end of the loop body contains a back-edge linking back to the condition block.
- A for loop connects its initializer to a condition block (**for_cond**), which branches to the body and merge blocks. The body links to an increment block (**for_incr**), which links back to the condition via a back-edge.
- A return statement immediately terminates a block, linking its block directly to the unique **EXIT** block. Subsequent statements placed after

an unconditional return within the same block are marked as dead, cut from CFG edges, and flagged as unreachable code.

6.2. Static Call Graph & Tarjan's SCC Cycle Detection

The CallGraph (**call_graph.py**) constructs a program-wide static call graph where nodes represent defined functions, and edges represent call sites (**CallExpr**).

It implements several robust queries:

- **Transitive Reachability:** Employs BFS/DFS to find all transitively reachable callees and reversed BFS to find transitively reaching callers.
- **Recursion Detection:** Detects recursive functions (both direct recursion $f \to f$ and mutual recursion $f \to g \to f$) by executing a cycle-detection pass on the call graph using **3-color DFS marking** (White: unvisited, Gray: visiting, Black: visited).
- **Tarjan's SCC Algorithm:** Detects and groups mutually recursive functions into Strongly Connected Components (SCCs) in $O(V + E)$ time.
- **Dead Function Detection:** Identifies dead functions that cannot be transitively reached from the program's unique entry point (main), reporting them as dead code.

---

7. Advanced IDE Features & Refactoring Engine (Phase 3)

The RefactoringEngine (**refactoring.py**) leverages the Symbol Table and the AST to deliver precise, editor-grade interactive features:

- **Go-to-Definition:** Translates a cursor coordinate (line, column) to its identifier token, queries the active Scope, and returns the exact coordinate SourceLocation of the definition site.
- **Find-All-References:** Uses a ReferenceCollector visitor to traverse the AST. It collects only those identifiers whose lexical name resolution points to the *exact same* Symbol instance (identity-match), preventing matching same-named variables in other scopes and ensuring scope-aware reference tracking.
- **Hover Documentation & Comments Extraction:** Returns the type signature of the hovered symbol. To fetch attached documentation comments (Javadoc/Doxygen) for functions, it tokenizes the source keeping comments (**keep_comments=True**), locates the declaration token, and scans **backwards** in the token stream. If the preceding non-formatting token is a **COMMENT_BLOCK** or **COMMENT_SINGLE**, it extracts its text.
- **Safe Rename Refactoring:** Receives a cursor location and a new_name. It performs:

1. **Conflict Check:** Raises an error if new_name is already declared in the local scope.
2. **Shadow Check:** Raises a "Shadow Conflict" error if renaming would shadow an outer declaration, or if any usage of the renamed variable is placed inside an inner scope that already defines **new_name** (accidental capture).
3. **Unified Diff Generation:** If safe, it replaces the lexeme at every collected reference site. The replacements are done in **reverse column order** on each line to preserve column offsets. It then uses Python's difflib to generate a standardized, git-compatible **Unified Diff** representation of the changes.

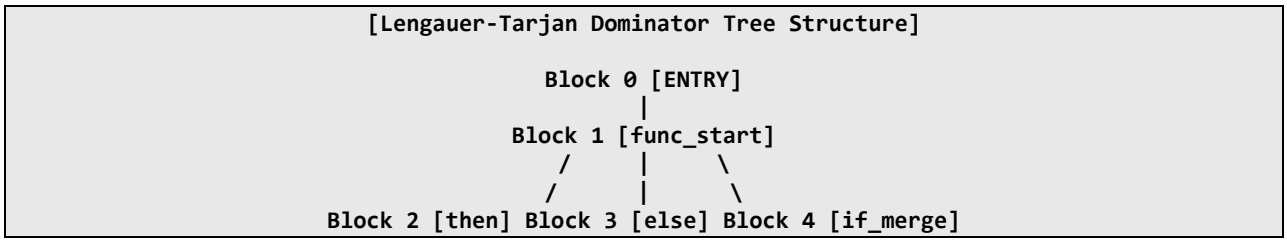
---

8. Advanced Compiler Bonus Features

To demonstrate advanced compiler engineering, we implemented and tested highly valuable bonus features:

8.1. Lengauer-Tarjan Dominator Tree

We implemented the mathematically complex **Lengauer-Tarjan algorithm** (**dominance.py**) to compute immediate dominators (idom) on the CFG basic blocks in $O(E\alpha(V))$ time using disjoint-set forests with path compression. From these dominators, we construct the Dominator Tree structure and compute the **Dominance Frontier (DF)** of each basic block (for join points, walking up the tree to idom and appending frontiers).



8.2. Static Single Assignment (SSA) Form

We implemented Cytron's worklist algorithm (**ssa.py**) to convert CFGs into **Static Single Assignment (SSA) form**:

1. **Phi Placement:** Uses the calculated Dominance Frontiers to place ϕ-functions at join points for variables that are redefined across paths.
2. **Variables Renaming:** Traverses the Dominator Tree in preorder. It renames variables with unique subscripts (x_1, x_2). It handles assignments and declarations by renaming the LHS with a new subscript while formatting the RHS with old subscripts (e.g. x_2 = x_1 + 1). To prevent recursion index corruption, the correct LHS subscript is saved in a local map (phi_lhs) at the start of block processing instead of

evaluating self.counters at the end of the recursion. It then populates ϕ arguments in successor blocks before popping subscripts to restore the scope context.

```
[SSA Basic Block Output Example]

Block 4 [if_merge]:
  - x_3 = phi(x_1, x_2)
  - int y_1 = x_3
```

---

9. Additional DevOps Bonuses

9.1. Docker Containerization

We containerized the entire system into a lightweight, secure Docker image (**Dockerfile**) based on python:3.12-slim. Building the image (**docker build -t cc-analyzer .**) and running it (**docker run -it cc-analyzer**) automatically boots the interactive CLI REPL console directly inside the container with no local installation or setup required.

9.2. Automated CI/CD Pipeline & Coverage Report

We configured a robust CI/CD pipeline using **GitHub Actions** (**.github/workflows/ci.yml**):

- Automatically triggers on every push or pull_request to the main branch.
- Installs dependencies and runs our 2**79 automated tests** while measuring code coverage using pytest-cov (guaranteeing/reporting >85% line coverage).
- Runs **generate_report.py** to highlight a canonical C file (**tests/canonical_test.c**) into an HTML page, which is then deployed automatically to **GitHub Pages** (accessible live at the repository URL).

9.3. Heuristic Language Classifier

We implemented a statistical heuristic language detector (**language_detector.py**) that classifies whether an unlabeled source code string is **C**, **Python**, or **Java** based on:

- Looking for shebang lines (identifying Python).
- Counting curly braces {} and semicolons ; (adding heavy weights to C/Java and penalizing Python).
- Counting statement-ending block colons : (identifying Python).
- Scanning keyword frequencies (e.g. struct for C, def for Python, public static void main for Java) and normalizing the final positive scores into ranked confidence percentages.

---

10. Interactive CLI REPL Console Guide

The system exposes all of its features and advanced analysis passes through an interactive command console (**repl.py**). Below is the complete command guide:

Command	Description	Example
load <code>	Loads raw C source code into the active engine.	load void main() { int x = 5; }
goto-def <line> <col>	Jumps to the declaration site of the target identifier.	goto-def 4 15
find-refs <line> <col>	Locates every scope-aware usage coordinate.	find-refs 2 9
hover <line> <col>	Shows variable type/function signature and attached doc comments.	hover 3 10
rename <line> <col> <name>	Safely renames symbol globally producing unified diff.	rename 2 9 new_name
show-cfg <func>	Displays CFG basic blocks, instructions and edges.	show-cfg factorial
show-callgraph	Displays program Call Graph and Tarjan’s SCC groups.	show-callgraph
show-dominators <func>	Displays Lengauer-Tarjan idom, DT tree, and DF.	show-dominators factorial
show-ssa <func>	Computes and displays the Static Single Assignment form.	show-ssa factorial
detect <code>	Predicts source language (C/Python/Java) with % score.	detect def f(): pass

Command	Description	Example
dead-code	Reports dead functions, unreachable blocks & unused vars.	dead-code
diagnostics	Prints accumulated lexical, syntactic, and semantic issues.	diagnostics
show-ast	Computes and displays the Abstract Syntax Tree (AST).	show-ast
show-symboltable	Computes and displays the Hierarchical Symbol Table & Scopes.	show-symboltable
help	Displays help usage information.	help
exit	Closes the REPL console.	exit

---

11. Test Suites Execution & Local Verification Guide

The compiler is backed by a **comprehensive testing suite consisting of 79 unit and integration tests** located inside **tests/** to guarantee correctness across all phases, graphs, refactoring engines, and bonuses.

11.1. Running the Test Suite

To run all automated tests and verify the front-end and middle-end pipelines locally inside your virtual environment:

```
python -m pytest
```

11.2. Generating Local HTML Coverage Reports

Our local **pytest.ini** automatically integrates with **pytest-cov** and configures coverage parameters. When you run pytest on your machine, it generates:

1. A detailed terminal command-line summary showing exact line numbers of uncovered code.
2. A standalone, interactive web report inside the **htmlcov/** folder.

To view your line-by-line coverage report locally on your machine (inspecting covered lines highlighted in green and uncovered lines highlighted in red):

```
# On Windows:
start htmlcov/index.html
# On macOS/Linux:
open htmlcov/index.html
```

11.3. Generating Physical Compilation Outputs

To process a custom C file (e.g., input.c) and generate all intermediate compilations into a dedicated output/ directory, run:

```
python main.py input.c
```

This automatically writes out:

- **output/tokens.txt** & **output/lexical_errors.txt**
- **output/syntax_errors.txt** & **output/parse_tree.txt** (ASCII AST)
- **output/ast.png** (Graphical dark-themed representation of the AST)
- **output/semantic_errors.txt** & **output/symbol_table.txt**
- **output/symbol_table.png** (ID-safe graphical representation of Scopes & Symbol tables)
- **output/call_graph.txt** & **output/call_graph.png** (Graphical representation of Call Graph)
- Function-level **output/cfg_*.txt** & **output/cfg_*.png** (Graphical basic blocks flowcharts)
- Function-level **output/dominator_tree_*.png** (Graphical Dominator Trees)
- Function-level **output/ssa_*.txt** & **output/ssa_*.png** (Graphical versioned SSA flowcharts with placed phi functions)

---

12. Conclusion

This project successfully demonstrates the construction of a complete, robust, and highly optimized compiler front-end and middle-end analysis layer for a C-subset from first principles. By grounding our implementation in the core theories studied throughout this course — regular languages, context-free grammars, attribute grammars, graph-based data-flow computations, and program transformations — we constructed a highly modular, professional-grade code analyzer.

The resulting tool integrates theoretical concepts (DFAs, recursive-descent parsing, lexical scoping, Tarjan’s SCC algorithm, Lengauer-Tarjan dominance,

and Cytron’s SSA conversions) into a unified, dockerized product backed by a fully automated CI/CD pipeline and 79 comprehensive tests. This system provides a solid foundation for further optimization passes, assembly code generation, or integration with modern text editors as an LSP language server.